

Convolutional Neural Networks

CSE427: Machine Learning

Lecture Roadmap

- 1 Why CNNs are needed
- 2 Image representation: height, width, channels
- 3 Convolution operation
- 4 Filters and feature maps
- 5 RGB convolution
- 6 Output dimension formula
- 7 Padding and stride
- 8 Parameter sharing
- 9 Pooling layer
- 10 Normalization layers
- 11 VGGNet-style architecture and parameter counting

Why Not Use Only Fully Connected Networks for Images?

Suppose an image has size:

$$224 \times 224 \times 3$$

Number of input values:

$$224 \times 224 \times 3 = 150,528$$

If one dense layer has 1000 neurons, then number of weights is:

$$150,528 \times 1000 = 150,528,000$$

Problem

Fully connected networks ignore spatial structure and require too many parameters.

CNNs solve this using:

local connectivity + parameter sharing

A CNN does not connect every input pixel to every neuron.
Instead, it uses a small filter:

$$3 \times 3, \quad 5 \times 5, \quad 11 \times 11$$

and slides it over the image.

small region $\rightarrow$ filter $\rightarrow$ one output value

As the filter slides, it produces a feature map.

Intuition

A filter acts like a feature detector. One filter may detect edges, another may detect curves, textures, or patterns.

A grayscale image has one channel:

$$H \times W \times 1$$

An RGB image has three channels:

$$H \times W \times 3$$

For example:

$$227 \times 227 \times 3$$

means:

227×227 red values

227×227 green values

227×227 blue values

Depth of RGB image = 3

Filter Shape Must Match Input Depth

If the input image is:

$$227 \times 227 \times 3$$

and we use an 11×11 filter, then the filter size is:

$$11 \times 11 \times 3$$

Why?

Because the filter must cover all input channels:

11×11 over Red

11×11 over Green

11×11 over Blue

Total weights per filter:

$$11 \times 11 \times 3 = 363$$

With one bias:

$$363 + 1 = 364 \text{ parameters}$$

For one 3×3 image patch and one 3×3 filter:

$$\text{output} = \sum_{i=1}^3 \sum_{j=1}^3 X_{ij} F_{ij}$$

That means:

multiply corresponding values

then:

add all results

If bias is used:

$$\text{output} = \sum_{i=1}^3 \sum_{j=1}^3 X_{ij} F_{ij} + b$$

Important

One filter position produces one number in the feature map.

Convolution Example: One Patch

Image patch:

$$X = \begin{bmatrix} 0.6 & 0.2 & 0.6 \\ 0.1 & -0.2 & -0.3 \\ -0.5 & -0.1 & -0.3 \end{bmatrix}$$

Filter:

$$F = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Convolution output:

$$\begin{aligned} & (0.6)(1) + (0.2)(1) + (0.6)(1) \\ & + (0.1)(0) + (-0.2)(0) + (-0.3)(0) \\ & + (-0.5)(-1) + (-0.1)(-1) + (-0.3)(-1) \\ & = 0.6 + 0.2 + 0.6 + 0 + 0 + 0 + 0.5 + 0.1 + 0.3 \\ & \boxed{= 2.3} \end{aligned}$$

What Does This Filter Detect?

The filter is:

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

It compares:

top row intensity

with:

bottom row intensity

If top is bright and bottom is dark, output is positive.

If top is dark and bottom is bright, output is negative.

This filter detects horizontal edge-like patterns.

Second Patch Example

Suppose the patch is:

$$X = \begin{bmatrix} -0.6 & -0.2 & -0.6 \\ -0.1 & 0.2 & 0.3 \\ 0.5 & 0.1 & 0.3 \end{bmatrix}$$

Using the same filter:

$$F = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Output:

$$\begin{aligned} & (-0.6)(1) + (-0.2)(1) + (-0.6)(1) \\ & + (-0.1)(0) + (0.2)(0) + (0.3)(0) \\ & + (0.5)(-1) + (0.1)(-1) + (0.3)(-1) \\ & = -0.6 - 0.2 - 0.6 - 0.5 - 0.1 - 0.3 \\ & \boxed{= -2.3} \end{aligned}$$

The sign changes because the edge direction changed.

From One Output to a Feature Map

A filter is not applied only once.
It slides over the whole image.
Each position produces one number.

Filter position 1 $\rightarrow$ one output

Filter position 2 $\rightarrow$ one output

Filter position 3 $\rightarrow$ one output

All outputs together form a feature map.

One filter produces one feature map.

If we use K filters:

We get K feature maps.

Suppose the input is:

$$64 \times 64 \times 1$$

and we use two filters:

$$3 \times 3 \times 1$$

Then each filter produces one feature map. Output depth equals number of filters:

$$D_2 = K$$

If:

$$K = 2$$

then output has depth:

$$2$$

$$64 \times 64 \times 1 \rightarrow 64 \times 64 \times 2$$

if padding keeps spatial size unchanged.

For an RGB image:

$$W \times H \times 3$$

a filter must also have depth 3:

$$F \times F \times 3$$

The filter has three parts:

$$F_R, \quad F_G, \quad F_B$$

The output is:

$$\sum R_{ij}F_{R,ij} + \sum G_{ij}F_{G,ij} + \sum B_{ij}F_{B,ij} + b$$

Key Point

One RGB filter produces one number at one spatial location after combining all channels.

Input:

$$227 \times 227 \times 3$$

Filter:

$$11 \times 11 \times 3$$

Weights per filter:

$$11 \times 11 \times 3 = 363$$

If we use 90 filters:

$$90 \times 363 = 32,670$$

Biases:

$$90$$

Total parameters:

$$32,670 + 90 = 32,760$$

Convolution parameters depend on filter size, depth, and number of filters.

Output Size Formula

Let input size be:

$$W_1 \times H_1 \times D_1$$

Filter size:

$$F \times F \times D_1$$

Number of filters K Stride S Padding P Then output size is:

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

$$H_2 = \frac{H_1 - F + 2P}{S} + 1$$

$$D_2 = K$$

$$\boxed{Output = W_2 \times H_2 \times K}$$

Output Dimension Example

Input image:

$$227 \times 227 \times 3$$

Filter:

$$11 \times 11 \times 3$$

Stride $S=4$ Padding $P=0$ Output width:

$$W_2 = \frac{227 - 11 + 2(0)}{4} + 1$$

$$W_2 = \frac{216}{4} + 1$$

$$W_2 = 54 + 1 = 55$$

Similarly:

$$H_2 = 55$$

If number of filters is 90:

$$\boxed{Output = 55 \times 55 \times 90}$$

Without padding:

$$P = 0$$

the filter cannot go outside the image boundary.

So the feature map becomes smaller.

Example:

$$227 \times 227$$

with:

$$11 \times 11, \quad S = 4$$

becomes:

$$55 \times 55$$

To keep size closer to original, we use padding.

Padding adds extra border values around the image.

Zero padding means adding zeros around the border.

If input is:

$$W_1 \times H_1$$

and padding is P , then effective size becomes:

$$(W_1 + 2P) \times (H_1 + 2P)$$

Output width:

$$W_2 = \frac{W_1 - F + 2P}{S} + 1$$

Why Use Padding?

- keeps spatial size from shrinking too quickly;
- allows border pixels to contribute more;
- helps build deeper CNNs.

Same Padding Example

Input:

$$32 \times 32 \times 3$$

Filter:

$$3 \times 3 \times 3$$

Stride:

$$S = 1$$

To keep output width 32, choose:

$$P = 1$$

Then:

$$W_2 = \frac{32 - 3 + 2(1)}{1} + 1$$

$$W_2 = 31 + 1 = 32$$

If $K = 10$ filters:

$$\boxed{\text{Output} = 32 \times 32 \times 10}$$

In a dense layer, different spatial locations use different weights.

In CNN, the same filter weights are reused at every location.

For one filter:

$$F \times F \times D_1$$

the same weights slide across the image.

One filter has one shared set of weights.

This gives two benefits:

- fewer parameters;
- same feature can be detected anywhere in the image.

For input:

$$227 \times 227 \times 3$$

Filter:

$$11 \times 11 \times 3$$

Each convolution position is like one neuron connected to:

$$11 \times 11 \times 3 = 363$$

input values.

For one filter, output size:

$$55 \times 55$$

So there are:

$$55 \times 55 = 3025$$

convolution positions.

But all these positions share the same:

For input:

$$227 \times 227 \times 3$$

Filter:

$$11 \times 11 \times 3$$

Each image block has:

$$11 \times 11 \times 3 = 363$$

values. Number of blocks:

$$55 \times 55 = 3025$$

Image matrix:

$$363 \times 3025$$

If there are 90 filters:

$$90 \times 363$$

Output:

$$\begin{aligned} &(90 \times 363)(363 \times 3025) \\ &= 90 \times 3025 \end{aligned}$$

This can be reshaped as $55 \times 55 \times 90$

Pooling reduces spatial size.

Common pooling:

2×2 max pooling with stride 2

It takes each 2×2 region and keeps the maximum value.

Example:

$$\begin{bmatrix} 1 & 3 \\ 2 & 5 \end{bmatrix} \rightarrow 5$$

Why Pooling?

- reduces computation;
- reduces memory;
- gives small translation robustness;
- keeps strongest feature response.

Max Pooling Example

Input feature map:

$$\begin{bmatrix} 1 & 3 & 2 & 4 \\ 5 & 6 & 1 & 2 \\ 0 & 1 & 7 & 8 \\ 3 & 2 & 5 & 9 \end{bmatrix}$$

Use:

2×2 max pooling, $S = 2$

Regions:

$$\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix} \rightarrow 6$$

$$\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix} \rightarrow 4$$

$$\begin{bmatrix} 0 & 1 \\ 3 & 2 \end{bmatrix} \rightarrow 3$$

$$\begin{bmatrix} 7 & 8 \\ 5 & 9 \end{bmatrix} \rightarrow 9$$

Output:

$$\boxed{\begin{bmatrix} 6 & 4 \\ 3 & 9 \end{bmatrix}}$$

Average pooling takes the average instead of maximum.

Example:

$$\begin{bmatrix} 1 & 3 \\ 5 & 6 \end{bmatrix}$$

Average:

$$\frac{1 + 3 + 5 + 6}{4} = \frac{15}{4} = 3.75$$

Average pooling keeps average information.

Comparison:

Max pooling: keeps strongest activation

Average pooling: keeps overall response

After convolution and pooling, we often have a 3D tensor.

Example:

$$8 \times 8 \times 20$$

Flatten converts it into a vector:

$$8 \times 8 \times 20 = 1280$$

So:

$$8 \times 8 \times 20 \rightarrow 1280 \text{ values}$$

Then a dense layer can classify:

$$1280 \rightarrow 10$$

for 10 classes.

Flatten prepares CNN features for a classifier.

Why Normalization is Needed

During training, layer inputs can change scale.

This may cause:

unstable gradients

slow convergence

sensitivity to initialization

Normalization helps by making activations more stable.

Benefits

- more stable training;
- faster convergence;
- less sensitivity to initialization;
- often allows higher learning rate;
- may improve generalization.

CNN mini-batch is often written as:

$$N \times C \times H \times W$$

where:

N = batch size

C = number of channels

H = height

W = width

Different normalization methods normalize over different axes.

The key difference is: which values are used to compute mean and variance?

Batch Normalization normalizes over:

$$N \times H \times W$$

for each channel C .

Meaning:

For each channel, use all batch images and all spatial locations.

Useful when:

batch size is reasonably large

Problem:

may perform poorly for very small batch size

Batch Norm depends on batch statistics.

Layer Normalization normalizes within one sample:

$$C \times H \times W$$

It does not depend on batch size.

Useful for:

RNNs

Transformers

variable batch size settings

Layer Norm is sample-wise, not batch-dependent.

Instance Normalization normalizes over:

$$H \times W$$

for each channel of each sample.

Useful for:

style transfer and image generation

Group Normalization divides channels into groups.

Then it normalizes over:

$$(C/G) \times H \times W$$

Useful when:

batch size is small

Group Norm is between Layer Norm and Instance Norm.

Method	Normalization Unit	Batch Dependent?
Batch Norm	$N \times H \times W$ per channel	Yes
Layer Norm	$C \times H \times W$ per sample	No
Instance Norm	$H \times W$ per channel per sample	No
Group Norm	$(C/G) \times H \times W$ per group	No

Choose normalization based on architecture and batch size.

VGGNet uses repeated blocks of:

3×3 convolution

with:

$$S = 1, \quad P = 1$$

and:

2×2 max pooling, $S = 2$

Convolution with $P = 1$ keeps spatial size same.

Pooling halves spatial size.

$$224 \rightarrow 112 \rightarrow 56 \rightarrow 28 \rightarrow 14 \rightarrow 7$$

VGG gradually reduces spatial size and increases channels.

Input:

$$224 \times 224 \times 3$$

First convolution:

$$3 \times 3, \quad K = 64$$

Parameter count:

$$(3 \times 3 \times 3) \times 64 + 64$$

$$= 27 \times 64 + 64$$

$$= 1728 + 64$$

$$\boxed{= 1792}$$

Output size:

$$224 \times 224 \times 64$$

Input:

$$224 \times 224 \times 64$$

Convolution:

$$3 \times 3, \quad K = 64$$

Parameter count:

$$(3 \times 3 \times 64) \times 64 + 64$$

$$= 576 \times 64 + 64$$

$$= 36,864 + 64$$

$$\boxed{= 36,928}$$

Output:

$$224 \times 224 \times 64$$

Why Fully Connected Layers Have Many Parameters

Before FC layer in VGG:

$$7 \times 7 \times 512$$

Flatten:

$$7 \times 7 \times 512 = 25,088$$

If FC layer has 4096 neurons:

$$25,088 \times 4096$$

$$= 102,760,448$$

weights.

Fully connected layers can dominate parameter count.

This is why modern CNNs often reduce or replace large FC layers.

Exam Practice Problem

An input image has size:

$$64 \times 64 \times 3$$

A convolution layer uses:

$$K = 16 \text{ filters}$$

Each filter has size:

$$5 \times 5$$

Stride:

$$S = 1$$

Padding:

$$P = 2$$

Find:

- 1 output dimension;
- 2 number of trainable parameters;
- 3 output dimension after 2×2 max pooling with $S = 2$.

Output width:

$$W_2 = \frac{64 - 5 + 2(2)}{1} + 1$$

$$W_2 = 63 + 1 = 64$$

Output height:

$$H_2 = 64$$

Depth:

$$D_2 = K = 16$$

So convolution output:

$$64 \times 64 \times 16$$

Parameters:

$$(5 \times 5 \times 3) \times 16 + 16$$

$$= 75 \times 16 + 16$$

$$= 1200 + 16 = 1216$$

After 2×2 max pooling:

$$32 \times 32 \times 16$$

- CNNs use filters to detect local patterns.
- One filter produces one feature map.
- Multiple filters produce multiple feature maps.
- Output size depends on input size, filter size, padding, and stride.
- RGB filters must cover all three channels.
- CNNs reduce parameters by parameter sharing.
- Pooling reduces spatial size.
- Normalization stabilizes training.
- VGGNet uses repeated 3×3 convolutions and 2×2 pooling.